

LegalLens — Project Documentation & Progress

AI-Powered Contract Intelligence Platform • Academic MCA Mini Project

IMPORTANT NOTICE / DISCLAIMER: *LegalLens is designed as an automated first-pass contract analysis tool for academic evaluation. It highlights risk indicators and enables grounded Q&A.; It does not provide legal advice and is never a substitute for a licensed attorney.*

1. Executive Summary & Problem Statement

Individuals, freelancers, and small businesses routinely execute contracts (employment agreements, freelance deliverables, rental leases, vendor scopes) without legal counsel due to high hourly legal fees and complex legalese. Generic AI chatbots provide unstructured, ungrounded, and unauditable text that risks legal exposure. **LegalLens** provides a fast, structured, explainable first-pass contract review coupled with grounded conversational question-answering strictly citing source clauses.

2. Project Constitution & Scope Discipline

Development is strictly governed by **AGENTS.md** (the project constitution). Per its directives:

- **Scope Discipline:** Built as an MCA mini project. Avoids Docker, Kubernetes, microservice splits, distributed message queues, or paid multi-cloud tiers. Prioritizes specification adherence over superficial enterprise complexity.
- **Clause-Level Traceability:** Every risk flag and risk score deduction must reference the specific clause ID and heuristic rule. Black-box or unexplained scores are prohibited.
- **Strict Grounded Q&A;:** Every answer must cite the specific retrieved clause(s) or explicitly state that the contract does not address the question.
- **User Account Isolation:** All documents, clauses, embeddings, and chat histories are scoped by user account.
- **Security:** No API keys or credentials committed; managed strictly via `.env` and `.env.example`.

3. Fixed Technology Stack

Component	Technology Selected	Rationale & Details
Backend Framework	FastAPI, Uvicorn, Python 3.10+	High performance, native async, automatic OpenAPI docs.
Frontend UI	React (Vite), Tailwind CSS	Modern, responsive dashboard, fast development cycle.
Database	PostgreSQL via SQLAlchemy (psycopg2)	Relational integrity for users, contracts, clauses, and risk reports.
Document Parsing	PyMuPDF (fitz) & python-docx	High-fidelity structural extraction for PDF and DOCX files.
Clause Classifier	InLegalBERT (law-ai/InLegalBERT)	Specialized legal language model fine-tuned on CUAD dataset.
Vector Store & RAG	Sentence-Transformers + ChromaDB	Local embedded persistent vector database with native metadata filtering.
Grounded LLM	External LLM API (Server-side)	Server-side calls only; protected API keys, strict citation rules.

4. Core 15 CUAD Classification Categories

From the 41 CUAD legal categories, the 15 most consequential for commercial and civil contracts were selected and mapped:

1. Termination	6. Governing Law	11. Warranties
2. Indemnification	7. Dispute Resolution	12. Exclusivity / Non-Solicit
3. Limitation of Liability	8. Intellectual Property	13. Severability
4. Confidentiality / NDA	9. Payment Terms	14. Force Majeure
5. Non-Compete	10. Term and Renewal	15. Assignment & Change of Control

5. Explainable Risk Assessment Engine

Rather than computing an uninterpretable average across all clauses (which dangerously dilutes severe issues), LegalLens employs an explainable **weighted-deduction model**:

Overall Risk Score: $Score = 100 - \sum_{i \in \text{flagged}} (Weight_i \times Confidence_{rule,i} \times Confidence_{classifier,i})$

- **Imbalance Heuristics:** Flags unilateral termination for convenience, uncapped indemnification, complete liability disclaimers, and indefinite non-competes.
- **Checklist Omission Audit:** Tailored checklists for Employment, Freelance, Rental, and Vendor agreements flag missing critical protections.
- **Auditability:** Every penalty points directly to clause index, triggering rule, penalty points deducted, and layman explanation.

6. Verified Implementation & Test Status (Phases 0–7)

Phase / Module	Test Suite File	Tests	Status & Pass Rate
Phase 0: Scaffold & Health	backend/tests/test_health.py	2	2 / 2 PASSED (100%)
Phase 1: Upload & Parsing	backend/tests/test_parser.py	6	6 / 6 PASSED (100%)
Phase 2: Clause Segmentation	backend/tests/test_segmenter.py	5	5 / 5 PASSED (100%)
Phase 3: Clause Classification	backend/tests/test_classifier.py	8	8 / 8 PASSED (100%)
Phase 4: Risk Assessment Engine	backend/tests/test_risk_engine.py	5	5 / 5 PASSED (100%)
Phase 5: Embedding & RAG QA	backend/tests/test_qa_engine.py	4	4 / 4 PASSED (100%)
Phase 6: Dashboard & API Wiring	backend/tests/test_api.py	7	7 / 7 PASSED (100%)
Phase 7: Datastore & Scoping	backend/tests/test_datastore.py	3	3 / 3 PASSED (100%)
TOTAL BACKEND SUITE	pytest backend/tests -v	40	40 / 40 PASSED (100%)
Frontend Production Build	npm run build (Vite + React)	1500 mod	SUCCESS (0 errors, 2.1s)

7. Master 10-Phase Project Roadmap

Phase	Title	Key Scope & Deliverables
Phase 0	Scaffold & Verification	[COMPLETED] Layout, venv, requirements, Vite/React skeleton, health test.
Phase 1	Document Upload & Parsing	[COMPLETED] PyMuPDF & DOCX parsing, structure preservation, rejection handling.
Phase 2	Clause Segmentation	[COMPLETED] Heading/numbering/paragraph-based clause splitting.
Phase 3	Clause Classification	[COMPLETED] 15 core CUAD categories, confidence scores, Colab notebook.
Phase 4	Risk Assessment Engine	[COMPLETED] Checklists, imbalance heuristics, weighted deduction model.
Phase 5	Embedding & RAG QA	[COMPLETED] Vector store, semantic search, grounded QA with citations.
Phase 6	Interactive Dashboard	[COMPLETED] React dashboard, upload flow, risk cards, chat, PDF export.
Phase 7	Data Store Wiring	[COMPLETED] PostgreSQL schema, user account isolation, cascade deletions.
Phase 8	Evaluation Harness	Precision/recall/macro-F1 on CUAD, risk accuracy, QA quality, Indian contract transfer.
Phase 9	Polish & Docs	Final docs, architecture diagrams, viva demo walkthrough script.